

Principles of Computer Science II

Algorithms for Bioinformatics

Marco Zecchini

Sapienza University of Rome

Lecture 2

What is an algorithm?

Pebble Game



- Game played in turns by 2 players.
- Two piles of equal number of pebbles.
- Each turn a player may either
 - take 1 pebble **from a single pile**, or
 - take 1 pebble **from both piles**.
- The player that takes the last pebble wins.

What is an algorithm?

Best Strategy for Winning the Pebble Game

- Does the first player always have an advantage?
- Let's consider the most simplified version.
 - Pebbles = 2 – we call this the 2×2 game.
 - Is there a winning strategy?
 - What is the winning strategy?

What is an algorithm?

Generalized Strategy for Winning the Pebble Game

- Can we generalize the strategy of the 2×2 game?
- What about the 3×3 game?
 - Consider different game sequences.
- Consider the $n \times n$ game.
 - Is there only one winning strategy?
 - How easy it is to describe our strategy?
 - Quality of solution.

What is an algorithm?

We build a matrix for all **game combinations** keeping track of the **winning moves**:

- ① ↑ take one pebble from pile A.
- ② ← take one pebble from pile B.
- ③ ↖ take one pebble from each pile.
- ④ * ignore move.

	0	1	2	3	4	5	6	7	8	9	10
0											
1											
2											
3											
4											
5											
6											
7											
8											
9											
10											

What is an algorithm?

- The first player always loses the 2×2 .
- Clearly also for 0×2 , 0×4 , ...
- Can we generalize for all games where each pile has an even number of pebbles?

	0	1	2	3	4	5	6	7	8	9	10
0	*		*		*		*		*		*
1											
2	*		*								
3											
4	*										
5											
6	*										
7											
8	*										
9											
10	*										

What is an algorithm?

- The first player always loses the 2×2 .
- Clearly also for 0×2 , 0×4 , ...
- Can we generalize for all games where each pile has an even number of pebbles?

	0	1	2	3	4	5	6	7	8	9	10
0	*		*		*		*		*		*
1											
2	*		*		*		*		*		*
3											
4	*		*		*		*		*		*
5											
6	*		*		*		*		*		*
7											
8	*		*		*		*		*		*
9											
10	*		*		*		*		*		*

What is an algorithm?

- Only 1 option for all 0×1 , 0×3 , ... and 1×0 , 3×0 , ...

	0	1	2	3	4	5	6	7	8	9	10
0	*		*		*		*		*		*
1											
2	*		*		*		*		*		*
3											
4	*		*		*		*		*		*
5											
6	*		*		*		*		*		*
7											
8	*		*		*		*		*		*
9											
10	*		*		*		*		*		*

What is an algorithm?

- Only 1 option for all $0 \times 1, 0 \times 3, \dots$ and $1 \times 0, 3 \times 0, \dots$

	0	1	2	3	4	5	6	7	8	9	10
0	*	←	*	←	*	←	*	←	*	←	*
1	↑										
2	*		*		*		*		*		*
3	↑										
4	*		*		*		*		*		*
5	↑										
6	*		*		*		*		*		*
7	↑										
8	*		*		*		*		*		*
9	↑										
10	*		*		*		*		*		*

What is an algorithm?

- Only 1 option for all 0×1 , 0×3 , ... and 1×0 , 3×0 , ...
- Can we generalize for other columns/rows where one pile has an odd number of pebbles and the other an even?

	0	1	2	3	4	5	6	7	8	9	10
0	*	←	*	←	*	←	*	←	*	←	*
1	↑		↑		↑		↑		↑		↑
2	*	←	*	←	*	←	*	←	*	←	*
3	↑		↑		↑		↑		↑		↑
4	*	←	*	←	*	←	*	←	*	←	*
5	↑		↑		↑		↑		↑		↑
6	*	←	*	←	*	←	*	←	*	←	*
7	↑		↑		↑		↑		↑		↑
8	*	←	*	←	*	←	*	←	*	←	*
9	↑		↑		↑		↑		↑		↑
10	*	←	*	←	*	←	*	←	*	←	*

What is an algorithm?

- Only 1 option for all 0×1 , 0×3 , ... and 1×0 , 3×0 , ...
- Can we generalize for other columns/rows where one pile has an odd number of pebbles and the other an even?
- What about the other rows/columns?

	0	1	2	3	4	5	6	7	8	9	10
0	*	←	*	←	*	←	*	←	*	←	*
1	↑	↖	↑	↖	↑	↖	↑	↖	↑	↖	↑
2	*	←	*	←	*	←	*	←	*	←	*
3	↑	↖	↑	↖	↑	↖	↑	↖	↑	↖	↑
4	*	←	*	←	*	←	*	←	*	←	*
5	↑	↖	↑	↖	↑	↖	↑	↖	↑	↖	↑
6	*	←	*	←	*	←	*	←	*	←	*
7	↑	↖	↑	↖	↑	↖	↑	↖	↑	↖	↑
8	*	←	*	←	*	←	*	←	*	←	*
9	↑	↖	↑	↖	↑	↖	↑	↖	↑	↖	↑
10	*	←	*	←	*	←	*	←	*	←	*

What is an algorithm?

An algorithmic approach for winning the Pebble Game

- How can we build the matrix for any game size, e.g., 20×20
- What is the algorithm for winning the game?

What is an algorithm?

An algorithmic approach for winning the Pebble Game

- How can we build the matrix for any game size, e.g., 20×20
- What is the algorithm for winning the game?
- Why in the world do I care about a game with two nerdy people and a bunch of rocks? I'm interested in biology, and this game has nothing to do with me

What is an algorithm?

An algorithmic approach for winning the Pebble Game

- How can we build the matrix for any game size, e.g., 20×20
- What is the algorithm for winning the game?
- Why in the world do I care about a game with two nerdy people and a bunch of rocks? I'm interested in biology, and this game has nothing to do with me
- It is the **sequence alignment** problem.
- The computational idea used to solve both problems is the same.

What is an algorithm?

An algorithmic approach for winning the Pebble Game

- How can we build the matrix for any game size, e.g., 20×20
- What is the algorithm for winning the game?
- Why in the world do I care about a game with two nerdy people and a bunch of rocks? I'm interested in biology, and this game has nothing to do with me
- It is the **sequence alignment** problem.
- The computational idea used to solve both problems is the same.
- We need to understand how algorithms work.

Methodology of solving a computational problem

- What is the problem at hand ?
 - Identify & Understand assumptions.
 - What is our goal ?
 - Identify similar problems/solutions in the bibliography
 - What are the theoretical foundation ?
 - Can we formulate the problem in a unambiguous and precise way ?
- What is the Input that we have ?
 - Do we have enough data or should we try to collect?
 - Open data sets ?
 - Can we synthesize input data ?
- What is the expected Output ?

Solution Sketch

- Do we have a rough idea of a solution ?
- Do we have identified an approach to solving the problem ?
 - think again !
 - go through the definition – maybe we overlooked something ?
- Write down a **solution sketch**
 - check if it adheres to the initial assumptions
 - can you try it out with a small input ?
- Is the solution correct ? can we provide some arguments ?
- What is the performance of the solution ?
- Can we think of a more efficient solution ?

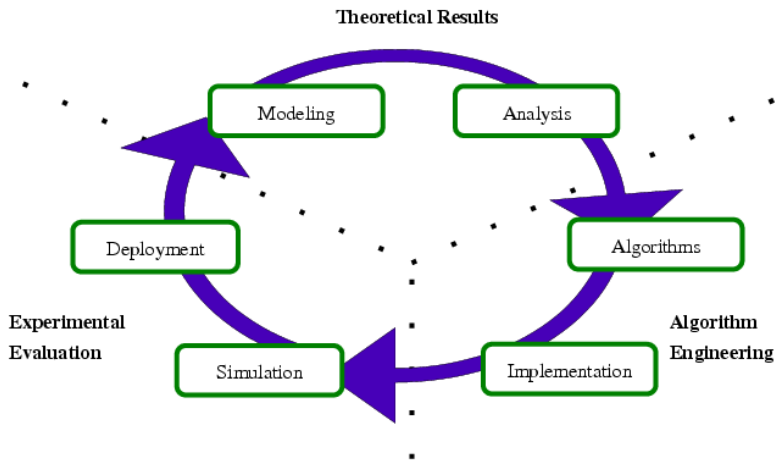
Implement the first version

- Pick your programming language of choice.
- Implement your solution
 - No need to try to make it elegant / fast.
- Get some input data
 - Open datasets
 - Small size
- Limited Evaluation
 - does it work ?
 - do you need to make any modifications ?
 - are there special cases that you missed ?

Iterative approach

- Step-by-step development
 - Continuous development.
 - Agile methodology.
- Identify issues in previous version
 - Code beautification.
 - Bug fixes.
 - Performance improvements.
 - Additional functionalities.
- Implement improvements
 - Make sure code is always clean + easy to maintain.
 - Keep detailed records of changes.
 - Always keep history of source code evolution.
- Performance Evaluation
 - bigger input.
 - scalability ?

Theoretical – Practical Approach Cycle



What is an algorithm?

Algorithm

An algorithm is a sequence of instructions that one must perform in order to solve a well-formulated **problem**. We will specify the problems in terms of their inputs and outputs.

What is an algorithm?

Algorithm

An algorithm is a sequence of instructions that one must perform in order to solve a well-formulated **problem**. We will specify the problems in terms of their inputs and outputs.

A well-formulated problem is unambiguous and precise, leaving no room for misinterpretation.

What is an algorithm?

Algorithm

An algorithm is a sequence of instructions that one must perform in order to solve a well-formulated **problem**. We will specify the problems in terms of their inputs and outputs.

A well-formulated problem is unambiguous and precise, leaving no room for misinterpretation.

An algorithm is the method to translate the inputs into the outputs.

What is pseudocode? (and why we use it)

- **Goal:** Describe an algorithm precisely without committing to a programming language.
- **Key primitives:**
 - Assignment: $a \leftarrow b$
 - Arithmetic: $+$, $-$, $\cdot$, $/$,
 - Conditionals: if A is true B else C
 - Loops: for $i \leftarrow a$ to b B / while A is true B
 - Arrays: $a = (a_1, \dots, a_n)$, access a_i
 - Subroutines: named blocks with arguments and return
- **How to read it:** treat each line as an atomic step; indentation = block structure.
- **How to use it:** first verify correctness by reasoning on inputs/outputs

Examples in pseudocode

MAX(a, b): returns the larger number

```
if a < b
    return b
else
    return a
```

Example:

MAX(7, 3) → returns 7

MAX(-1, 4) → returns 4

ADDUNTIL(b): smallest i s.t.
 $1 + 2 + \dots + i > b$

```
i ← 1
total ← i
while total ≤ b
    i ← i + 1
    total ← total + i
return i
```

Example: ADDUNTIL(25) → returns 7 (since $1+2+3+4+5+6 = 21 \leq 25$, $1+\dots+7 = 28 > 25$)

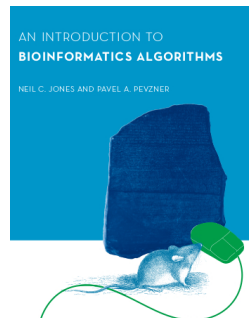
Observation: Pseudocode describes the algorithm's logic independently of any programming language — indentation shows structure and flow.

Is this algorithm good?

- We have identified a problem...
- ... we came out with an algorithm that solve the problem (our course)...
- ... but **does the algorithm solve the problem?** and **at which cost?** **Are there better solutions?** (again, our course)

Does the algorithm solve the problem?

Jones, Pevzner: An Introduction to
Bioinformatics Algorithms. MIT Press,
2004



Section 2.3 - 2.4, the US changing problem.

Does the algorithm solve the problem?

United States Change Problem:

Convert some amount of money into the fewest number of coins.

Input: An amount of money, M , in cents.

Output: The smallest number of quarters q , dimes d , nickels n , and pennies p whose values add to M (i.e., $25q + 10d + 5n + p = M$ and $q + d + n + p$ is as small as possible).

USCHANGE(M)

```
1   $r \leftarrow M$ 
2   $q \leftarrow r/25$ 
3   $r \leftarrow r - 25 \cdot q$ 
4   $d \leftarrow r/10$ 
5   $r \leftarrow r - 10 \cdot d$ 
6   $n \leftarrow r/5$ 
7   $r \leftarrow r - 5 \cdot n$ 
8   $p \leftarrow r$ 
9  return ( $q, d, n, p$ )
```

Does the algorithm solve the problem?

Change Problem:

Convert some amount of money M into given denominations, using the smallest possible number of coins.

Input: An amount of money M , and an array of d denominations $\mathbf{c} = (c_1, c_2, \dots, c_d)$, in decreasing order of value ($c_1 > c_2 > \dots > c_d$).

Output: A list of d integers $i_1, i_2, \dots, i_d$ such that $c_1 i_1 + c_2 i_2 + \dots + c_d i_d = M$, and $i_1 + i_2 + \dots + i_d$ is as small as possible.

BETTERCHANGE($M, \mathbf{c}, d$)

```
1   $r \leftarrow M$ 
2  for  $k \leftarrow 1$  to  $d$ 
3       $i_k \leftarrow r / c_k$ 
4       $r \leftarrow r - c_k \cdot i_k$ 
5  return  $(i_1, i_2, \dots, i_d)$ 
```

Does the algorithm solve the problem?

If $M = 40$ and $\mathbf{c} = (25, 10, 5, 1)$,
BetterChange returns $(1, 1, 1, 0)$ while
 $(2, 0, 0, 0)$ would be the right solution.

```
BETTERCHANGE( $M, \mathbf{c}, d$ )  
1   $r \leftarrow M$   
2  for  $k \leftarrow 1$  to  $d$   
3       $i_k \leftarrow r/c_k$   
4       $r \leftarrow r - c_k \cdot i_k$   
5  return  $(i_1, i_2, \dots, i_d)$ 
```

Does the Algorithm solve the problem?

```
BRUTEFORCECHANGE( $M, \mathbf{c}, d$ )
1   $smallestNumberOfCoins \leftarrow \infty$ 
2  for each  $(i_1, \dots, i_d)$  from  $(0, \dots, 0)$  to  $(M/c_1, \dots, M/c_d)$ 
3       $valueOfCoins \leftarrow \sum_{k=1}^d i_k c_k$ 
4      if  $valueOfCoins = M$ 
5           $numberOfCoins \leftarrow \sum_{k=1}^d i_k$ 
6          if  $numberOfCoins < smallestNumberOfCoins$ 
7               $smallestNumberOfCoins \leftarrow numberOfCoins$ 
8               $bestChange \leftarrow (i_1, i_2, \dots, i_d)$ 
9  return ( $bestChange$ )
```

Does the Algorithm solve the problem?

```
BRUTEFORCECHANGE( $M, \mathbf{c}, d$ )
1   $smallestNumberOfCoins \leftarrow \infty$ 
2  for each  $(i_1, \dots, i_d)$  from  $(0, \dots, 0)$  to  $(M/c_1, \dots, M/c_d)$ 
3       $valueOfCoins \leftarrow \sum_{k=1}^d i_k c_k$ 
4      if  $valueOfCoins = M$ 
5           $numberOfCoins \leftarrow \sum_{k=1}^d i_k$ 
6          if  $numberOfCoins < smallestNumberOfCoins$ 
7               $smallestNumberOfCoins \leftarrow numberOfCoins$ 
8               $bestChange \leftarrow (i_1, i_2, \dots, i_d)$ 
9  return  $(bestChange)$ 
```

Yes

Does the Algorithm solve the problem?

```
BRUTEFORCECHANGE( $M, \mathbf{c}, d$ )
1   $smallestNumberOfCoins \leftarrow \infty$ 
2  for each  $(i_1, \dots, i_d)$  from  $(0, \dots, 0)$  to  $(M/c_1, \dots, M/c_d)$ 
3       $valueOfCoins \leftarrow \sum_{k=1}^d i_k c_k$ 
4      if  $valueOfCoins = M$ 
5           $numberOfCoins \leftarrow \sum_{k=1}^d i_k$ 
6          if  $numberOfCoins < smallestNumberOfCoins$ 
7               $smallestNumberOfCoins \leftarrow numberOfCoins$ 
8               $bestChange \leftarrow (i_1, i_2, \dots, i_d)$ 
9  return  $(bestChange)$ 
```

Yes but at which cost?

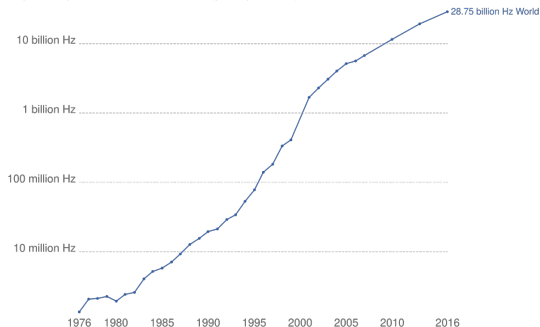
Measuring Performance

- Performance of an algorithm?
 - Speed/Computational Time
 - Memory/Space
 - Robustness/Failures
 - Network/Communication
 - Consumption/Energy
 - ...
- How can we measure the speed/memory/robustness/... of an algorithm?
- How much performance degrades when the amount of input data increases?

Is not CPU enough?

Microprocessor clock speed

Microprocessor clock speed measures the number of pulses per second generated by an oscillator that sets the tempo for the processor. It is measured in hertz (pulses per second).



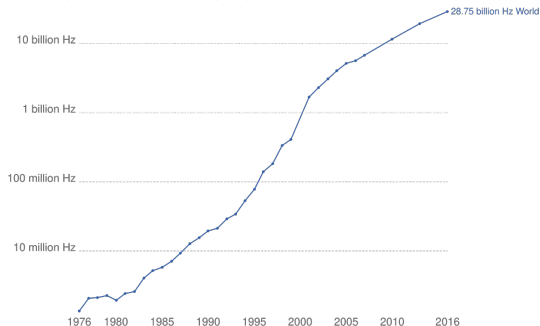
Source: Ray Kurzweil (2005, updated to 2016). The Singularity Is Near: When Humans Transcend Biology.

CC BY

Is not CPU enough?

Microprocessor clock speed

Microprocessor clock speed measures the number of pulses per second generated by an oscillator that sets the tempo for the processor. It is measured in hertz (pulses per second).



Source: Ray Kurzweil (2005, updated to 2016). The Singularity Is Near: When Humans Transcend Biology.

CC BY

No! We want to be independent

Computational Time Complexity

Computational Complexity

Describes the change in the runtime of an algorithm, depending on the change in the input data's size.

- Measures the speed of an algorithm.
- How much additional time it requires when the amount of input data increases.
- Examples:
 - How much longer does it take to find an element within an unsorted array when the size of the array doubles?
 - How much longer does it take to find an element within a sorted array when the size of the array doubles?

Space Complexity

Computational Complexity

Describes the requirements in terms of memory of an algorithm, depending on the size of the input data.

- Measures the memory requirements of an algorithm.
- Without considering the size of the input data.
- Additional memory is used by:
 - Helper variables within loops.
 - Temporary data structures.
 - Call stack.
 - ...

Complexity Classes – Big O Notation

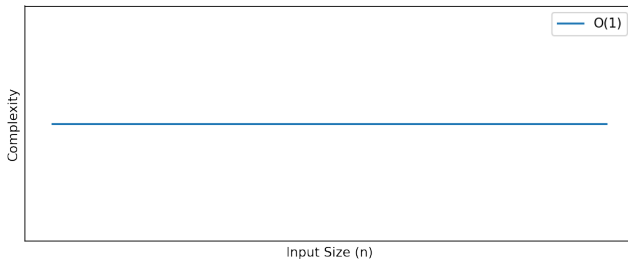
- We organize algorithms into **Complexity Classes**
- A complexity class is noted using the Bachmann-Landau symbol $\mathcal{O}$ ("big O")
- Let f the function to be estimated
- Let g the comparison function
- We write $f(x) = \mathcal{O}(g(x))$ as $x \rightarrow \infty$
- f is bounded above by g (up to constant factor) asymptotically.
- We do not measure the **exact running time** rather we classify the behaviour when n is **sufficiently large**.

Complexity Classes – Asymptotic behaviour

- An algorithm may contain sub-components of different complexity.
- For large inputs, the behaviour will be dominated by the part of the complexity that grows fastest.
 - Complexity function $g(n) = 100 \times n^2 + 10000 \times n + 840$ grows like $\mathcal{O}(n^2)$
 - Complexity function $g(n) = 0.33 \times n^3$ grows like $\mathcal{O}(n^3)$
- If $f(x)$ is a sum of several terms: we keep the one with the largest growth rate.
- If $f(x)$ is a product of several factors, any constants can be omitted.

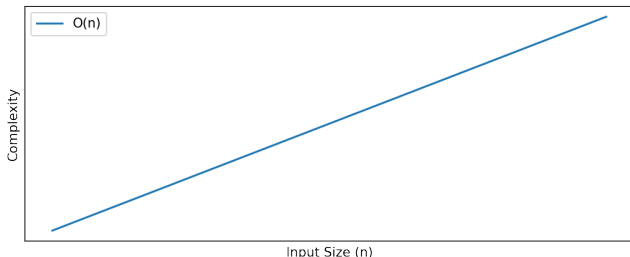
Constant Time – $\mathcal{O}(1)$

- Pronounced: “Order 1”, “O of 1”, “big O of 1”
- The runtime is constant.
- Independent of the number of input elements n .
- Examples
 - Accessing a specific element of an array of size n .
 - Inserting an element at the beginning of a list.



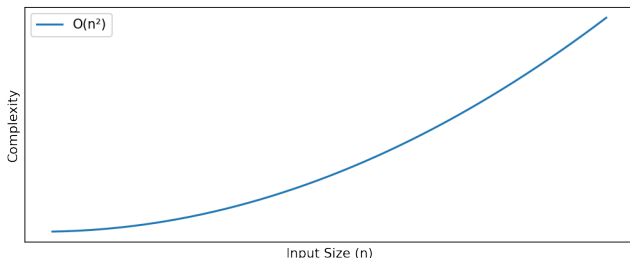
Linear Time – $\mathcal{O}(n)$

- Pronounced: “Order n ”, “O of n ”, “big O of n ”
- Runtime grows linearly with the number of input elements n .
- If n doubles, then the runtime approximately doubles, too.
- Examples
 - Finding a specific element in an array of size n .
 - Summing up all elements of an array.



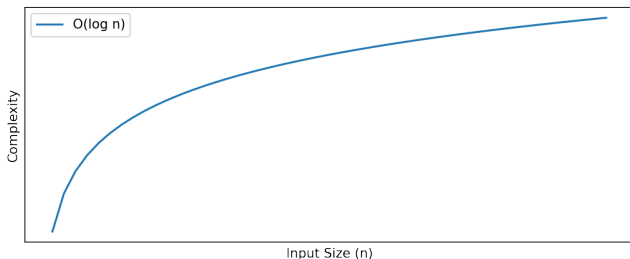
Quadratic Time – $\mathcal{O}(n^2)$

- Pronounced: “Order n squared”, “O of n squared”, “big O of n squared”
- Runtime grows linearly to the square of the number of input elements n .
- If n doubles, then the runtime approximately quadruples.
- Examples
 - Simple sorting algorithms (e.g., Insertion Sort).



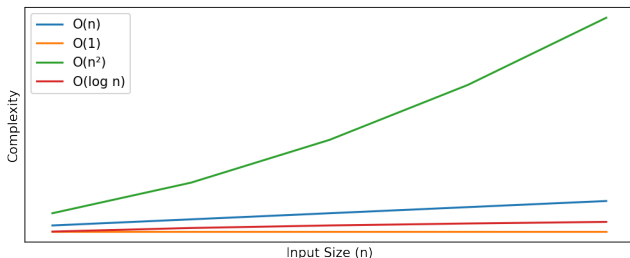
Logarithmic Time – $\mathcal{O}(\log n)$

- Pronounced: “Order log n ”, “O of log n ”, “big O of log n ”
- Runtime increases by a constant amount when the number of input elements n doubles.
- Examples
 - Binary search.



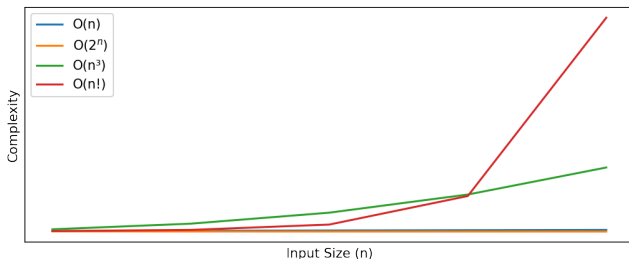
Big O Notation Order

- $\mathcal{O}(1)$ – constant time
- $\mathcal{O}(\log n)$ – logarithmic time
- $\mathcal{O}(n)$ – linear time
- $\mathcal{O}(n \log n)$ – quasilinear time
- $\mathcal{O}(n^2)$ – quadratic time



Other Complexity Classes

- $\mathcal{O}(n^m)$ – polynomial time
- $\mathcal{O}(2^n)$ – exponential time
- $\mathcal{O}(n!)$ – factorial time
- ...



Example: SUMINTEGERS(n) — code & complexity side-by-side

Pseudocode

```
SUMINTEGERS( $n$ )  
  sum  $\leftarrow$  0  
  for  $i \leftarrow 1$  to  $n$   
    sum  $\leftarrow$  sum +  $i$   
  return sum
```

Task: Compute $1 + 2 + \dots + n$ using a loop.

Complexity analysis

- The loop runs exactly n times.
- Each iteration performs:
 - one addition,
 - one assignment.
- Total number of basic operations: proportional to n .
- **Time:**
 $T(n) = c \cdot n + k = O(n)$
- **Space:** $S(n) = O(1)$ (only two variables).

Example: ADDUNTIL(b) — code & complexity side-by-side

Pseudocode

ADDUNTIL(b)

$i \leftarrow 1$

$total \leftarrow i$

 while $total \leq b$

$i \leftarrow i + 1$

$total \leftarrow total + i$

 return i

Task: return the smallest i such
that $1 + 2 + \dots + i > b$.

Complexity analysis

- After k iterations:
$$1 + 2 + \dots + k = \frac{k(k+1)}{2}.$$
- Loop stops at the smallest k with $\frac{k(k+1)}{2} > b$.
- Solve the inequality:
$$\frac{k^2 + k - 2b > 0 \Rightarrow k \approx \frac{\sqrt{8b+1} - 1}{2}.$$
- Therefore, the number of iterations is $\mathcal{O}(\sqrt{b})$ and each iteration is $\mathcal{O}(1)$.
- **Time:** $T(b) = \mathcal{O}(\sqrt{b})$
Space: $S(b) = \mathcal{O}(1)$.

Little-oh and Big-Theta notations

- We write $f(x) = o(g(x))$ – read “ $f(x)$ is little-oh of $g(x)$ ”
 - $g(x)$ grows much faster than $f(x)$
 - f is dominated by g asymptotically.
 - $\mathcal{O}$ has to be true for at least one constant M , little-o holds for **every** positive constant ϵ , however small.
- We write $f(x) = \Theta(g(x))$ – read “ $f(x)$ is big-theta of $g(x)$ ”
 - f is bounded both above and below by g asymptotically.
- Consider $T(n) = 73n^3 + 22n^2 + 58$, all the following are generally acceptable:
 - $T(n) = \mathcal{O}(n^{100})$ – grows asymptotically no faster than n^{100}
 - $T(n) = \mathcal{O}(n^3)$ – grows asymptotically no faster than n^3
 - $T(n) = \Theta(n^3)$ – grows asymptotically as fast as n^3

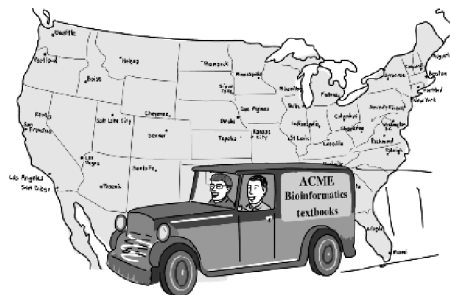
Tractable vs Intractable Problems

- Some problems cannot be solved
- US Change Problem (subset sum problem (SSP)) can only be solved in exponential time (intractable)

Complexity Theory

- Problems are categorized into classes (exponential time problems, polynomial time problems) since '70s
- Special class: NP-Complete problems are problems that appear to be difficult (no poly algorithm) but there is no proof of it.
- If one NP-Complete problem is solved, then all the others with almost the same algorithm

NP-Complete problem



Traveling Salesman Problem:

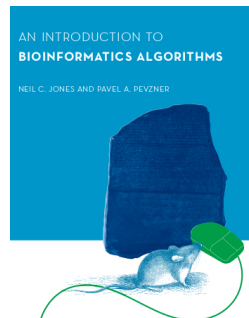
Find the shortest path through a set of cities, visiting each city only one time.

Input: A map of cities, roads between the cities, and distances along the roads.

Output: A sequence of roads that will take a salesman through every city on the map, such that he will visit each city exactly once, and will travel the shortest total distance.

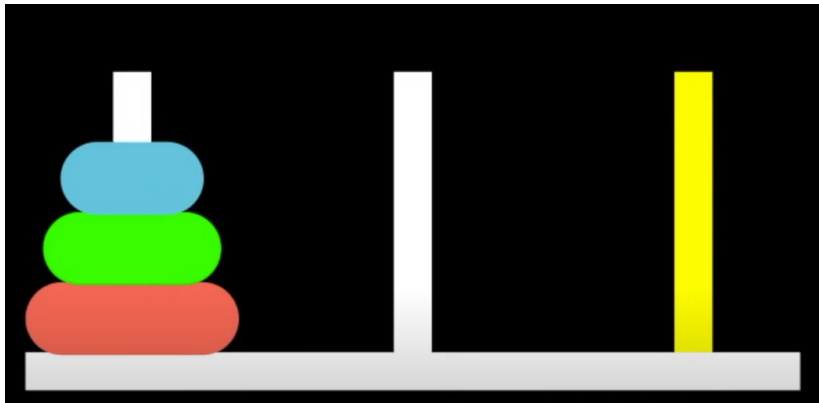
Tower of Hanoi problem

Jones, Pevzner: An Introduction to
Bioinformatics Algorithms. MIT Press,
2004



Section 2.5, the Tower of Hanoi problem.

Tower of Hanoi problem



<https://www.youtube.com/watch?v=rf6uf3jNjbo>

Tower of Hanoi Solution

```
HANOITOWERS(n, fromPeg, toPeg)
1  if n = 1
2      output "Move disk from peg fromPeg to peg toPeg"
3      return
4  unusedPeg ← 6 - fromPeg - toPeg
5  HANOITOWERS(n - 1, fromPeg, unusedPeg)
6  output "Move disk from peg fromPeg to peg toPeg"
7  HANOITOWERS(n - 1, unusedPeg, toPeg)
8  return
```

Recursion Coding Style

Recursion is a way of programming or coding a problem, in which a function calls itself one or more times in its body. Usually, it is returning the return value of this function call. If a function definition fulfils the condition of recursion, we call this function a recursive function.

Termination condition:

- A recursive function has to terminate to be used in a program.
- A recursive function terminates, if with every recursive call the solution of the problem is downsized and moves towards a base case.
- A base case is a case, where the problem can be solved without further recursion.

Factorial Computation: Using Iteration

```
def iterative_factorial(n):  
    result = 1  
    for i in range(2,n+1):  
        result *= i  
    return result
```

Factorial Computation: Using Recursion

```
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Factorial Computation

```
def factorial(n):  
    print("factorial has been called with n = " + str(  
        n))  
    if n == 1:  
        return 1  
    else:  
        res = n * factorial(n-1)  
        print("intermediate result for ", n, " *  
            factorial(" ,n-1, "): ",res)  
        return res  
  
print(factorial(5))
```

Fibonacci Numbers

The Fibonacci numbers are defined by:

$$F_n = F_{n-1} + F_{n-2}$$

where $F_0 = 0$ and $F_1 = 1$

- 0,1,1,2,3,5,8,13,21,34,55,89, ...

Factorial Computation: Using Recursion

```
def fib(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fib(n-1) + fib(n-2)
```

Factorial Computation: Using Iteration

```
def fibi(n):  
    a, b = 0, 1  
    for i in range(n):  
        a, b = b, a + b  
    return a
```

Measure Performance

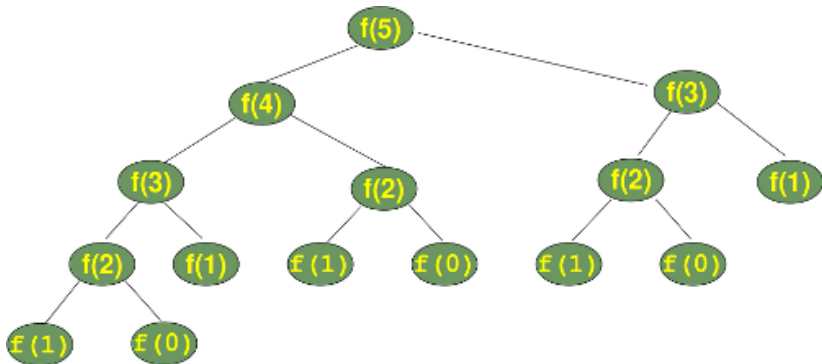
```
import time

for i in range(1,41):
    t1 = time.perf_counter()
    s = fib(i)
    t2 = time.perf_counter() - t1

    t3 = time.perf_counter()
    s = fibi(i)
    t4 = time.perf_counter() - t3

    print(f"n={i}, fib: {t2:.2f}, fibi: {t4:.2f},
          percent: {t2/t4:.2f}")
```

Fibonacci Numbers



Factorial Computation: Using Recursion and Memory

```
memo = {0:0, 1:1}
def fibm(n):
    if not n in memo:
        memo[n] = fibm(n-1) + fibm(n-2)
    return memo[n]
```